

# {css}

**Clase 04**



**CONQUER BLOCKS**

# Clase anterior

**Clase anterior**

**Tipos de etiquetas y CSS del navegador**

**Etiquetas de bloque**

---

**Etiquetas de línea**

---

**Navegadores y motores de render**

---

**Inspector de elementos a fondo**

---

**CSS por defecto**

---

**Normalizar o Restablecer CSS**

---

# <Índice>

## Clase 04

### Clase 4: Cascada y prioridad

Normalizar o Restablecer CSS

---

¿Qué significa hojas de estilo en cascada?

---

Cascada en CSS

---

Herencia

---

Especificidad

---

!important

---



# Normalizar o Restablecer CSS



# Normalizar o Restablecer CSS

DEMO

# Normalizar o Restablecer CSS

Mi consejo  
Combinación entre **Normalize** y **Reset**  
para lo no establecido

**¿Qué significa hojas de estilo  
en cascada?**



## ¿Por qué se llaman estilos en cascada?

Las propiedades de estilo pueden estar escritas en varios sitios (en varios lugares de la página web o de la hoja de estilo) y dependiendo del sitio, afectan a más o menos elementos.

## ¿Por qué se llaman estilos en cascada?

Cuando un elemento está contenido en otro (por ejemplo, un párrafo `<p>` dentro de una división `<div>`), al elemento de dentro se le aplican también las propiedades definidas para el elemento de fuera (al párrafo `<p>` se le aplicarían las propiedades definidas para la división `<div>`).

## ¿Por qué se llaman estilos en cascada?

Dos reglas distintas pueden ser de aplicación a un mismo elemento (por ejemplo, a un párrafo `<p>` con clase nombre, le es de aplicación tanto el selector `.nombre` como el selector `p`).



# CSS

Si las propiedades (escritas en diferentes sitios o para diferentes elementos) no entran en conflicto, el navegador aplica todas las propiedades. Por ejemplo, si el color de fondo de un elemento está definido en un sitio y el tamaño de letra en otro sitio, el navegador aplicará ambas propiedades al elemento.

Pero si las propiedades entran en conflicto (por ejemplo, el color del fondo del elemento está definido en varios sitios con colores distintos), existen reglas para decidir qué propiedad tiene preferencia.

# CSS

En las hojas de estilos complejas, es habitual que varias reglas CSS se apliquen a un mismo elemento HTML. El problema de estas reglas múltiples es que se pueden dar **colisiones** como la del siguiente ejemplo:

```
p { color: red; }
```

```
p { color: blue; }
```

```
<p>...</p>
```

¿De qué color se muestra el párrafo anterior?



CSS

DEMO

# CSS

CSS tiene un mecanismo de resolución de colisiones muy complejo y que tiene en cuenta el tipo de hoja de estilo que se trate (**de navegador, de usuario o de diseñador**), la importancia de cada regla y lo específico que sea el selector.

# CSS

El método seguido por CSS para resolver las colisiones de estilos se muestra a continuación:

1. Determinar todas las declaraciones que se aplican al elemento para el medio CSS seleccionado.
2. Ordenar las declaraciones según su origen (CSS de navegador, de usuario o de diseñador) y su prioridad (palabra clave !important).



# CSS

El método seguido por CSS para resolver las colisiones de estilos se muestra a continuación:

3. Ordenar las declaraciones según lo específico que sea el selector. Cuanto más genérico es un selector, menos importancia tienen sus declaraciones.
4. Si después de aplicar las normas anteriores existen dos o más reglas con la misma prioridad, se aplica la que se indicó en último lugar.

Hasta que no se expliquen más adelante los conceptos de tipo de hoja de estilo y la prioridad, el **mecanismo simplificado** que se puede aplicar es el siguiente:

1. Cuanto más específico sea un selector, más importancia tiene su regla asociada.
2. A igual especificidad, se considera la última regla indicada.



# Cascada en CSS

# Cascada en CSS

Cuando hablamos de la Cascada CSS nos referimos al algoritmo o conjunto de reglas y normas que tiene el navegador para aplicar estilos CSS a un elemento HTML.

No es tan sencillo como la mayoría cree, ya que hay múltiples casos donde pueden existir ambigüedades o conflictos de valores, y el navegador debe resolverlos para saber cual debe prevalecer.

# Cascada en CSS

Imagina que tenemos enlazados dos archivos .css mediante `<link>` y en cada uno tenemos un mismo selector `.text` aplicando colores diferentes. ¿Cuál de los dos se aplicará finalmente? Quizás pienses que el último que haya sido definido. A veces, esto es cierto, pero otras muchas veces no.

**Expliquemos la razón.**



# Cascada en CSS

La cascada comprende varios **temas importantes** a la hora de resolver dichos conflictos:

- **Importancia:** Los estilos pueden tener una importancia concreta, eso le da mayor o menor prioridad.
- **Origen:** Existen varios orígenes de CSS, cada uno con una prioridad específica.
- **Orden de aparición:** El orden en el que se procesa el CSS influye a la hora de resolver conflictos.
- **Especificidad:** Cuanto más específicos sean los estilos CSS, mayor prioridad.

# Cascada en CSS

## Importancia

# Navegadores y motores de render

## Importancia

Existe la posibilidad de añadir el texto !important al final de una regla, consiguiendo que el navegador le de prioridad a la hora de evaluar dichos estilos sobre otros.



# Navegadores y motores de render

## Importancia

```
<div class="text">Texto del elemento</div>

<style>
.text {
  color: red!important;
}

.text {
  color: blue;
}
</style>
```

# Cascada en CSS

Origen



# Navegadores y motores de render

## Origen

La cascada tiene en cuenta las hojas de estilo donde está colocado nuestro código CSS. Generalmente, no necesitaremos preocuparnos de este factor, ya que nos centraremos casi siempre en el CSS de autor. Sin embargo, es una buena idea conocer en que se basa cada origen.

# Navegadores y motores de render

## Origen

- Hoja de estilo de agente de usuario: Estilos CSS que tiene y aplica el navegador por defecto.
- Hoja de estilos de usuario: Estilos CSS que añade el usuario para personalizar, por ejemplo, con UserStyles.
- Hoja de estilo de autor/diseñador: Son los estilos CSS que crea el desarrollador en la página web.

# Cascada en CSS

## Orden de aparición



# Cascada en CSS

## Orden de aparición

Esta suele ser una de las partes de la cascada que más clara suele estar, ya que es bastante intuitiva. En el tema introductorio Cómo enlazar CSS, vimos que es posible aplicar estilos CSS a un documento de varias formas:

- [1] Atributo style en una etiqueta HTML.
- [2] Bloque de estilos <style> en el documento HTML.
- [3] Archivo .css externo referenciado mediante <link rel="stylesheet">.

# Cascada en CSS

**Orden de aparición**

Demo

# Cascada en CSS

## Especificidad



# Navegadores y motores de render

## Especificidad

En el caso de que tengamos un conflicto donde no se haya eliminado la ambigüedad, se pasa a calcular el selector más específico siguiendo unas normas que se detallan a continuación:

1. Componente Id
2. Componente Clase
3. Componente Elemento

# Navegadores y motores de render

## Especificidad (para el futuro)

```
div { ... } /* 0,0,1 (1 elemento) */
div div { ... } /* 0,0,2 (2 elementos) */
#pagina div { ... } /* 1,0,1 (1 id y 1 elemento) */
#pagina div:hover { ... } /* 1,1,1 (1 id, 1 pseudoclase y 1 elemento) */
#pagina div:hover a { ... } /* 1,1,2 (1 id, 1 pseudoclase y 2 elementos) */
#pagina .sel:hover>a { ... } /* 1,2,1 (1 id, 1 clase, 1 pseudoclase y 1 elemento) */
```

## Calculadora



# Herencia

# Herencia

Es uno de los aspectos, junto con la cascada, más infravalorados a la hora de aprender CSS

# Herencia

Debemos saber que algunas propiedades CSS, como color o font-family, se heredan desde los elementos HTML padres a los elementos HTML hijos, modificando el valor que tienen por defecto.



# Herencia

```
<div class="container">
  <p>Texto del padre</p>
  <div class="child">
    <p>Texto del hijo</p>
  </div>
</div>

<style>
  .container {
    color: green;          /* Propiedad heredable */
    border: 1px solid red; /* Propiedad no heredable */
  }
</style>
```

# Herencia

Si observas el resultado, verás que tanto el texto de `.container` (padre) como el texto de `.child` (hijo) aparecerán de color verde. Esto en cierta forma debería resultarnos lógico, ya que el elemento `.child` se encuentra en el interior de `.container`, y no hay ninguna otra regla más específica que le dé otro color de texto a ese elemento interior.

# Herencia

Sin embargo, con el borde rojo sólo se aplica borde al .container. El elemento .child no se ve afectado. Esto ocurre porque algunas propiedades como color o font-family se heredan en los hijos, mientras que otras propiedades como border, no.



# Herencia

Si esta propiedad aplicara herencia a sus hijos, todos los elementos HTML situados en el interior de `.container` tendrían su propio borde rojo, comportamiento que no suele ser el deseado. Por esa razón, la herencia no ocurre con todas las propiedades CSS, sino sólo con algunas propiedades donde si suele ser deseable.

# Herencia

## Valores especiales de herencia

Además de los valores habituales de cada propiedad CSS que iremos viendo a lo largo de nuestros artículos, también podemos aplicar ciertos valores especiales que son comunes a todas las propiedades CSS existentes. Con estos valores modificamos el comportamiento de la herencia en dicha propiedad



# Herencia

## Valores especiales de herencia

- **inherit**: La propiedad hereda el valor que tiene la misma propiedad CSS en su elemento padre.
- **initial**: Establece el valor al valor inicial definido por la especificación CSS.
- **unset**: Resetea el valor. En propiedades heredables, actúa como inherit, en otro caso, como initial.
- **revert**: Similar a unset, salvo que haya definido un origen de agente o de usuario (ver más adelante).

# Herencia

## Valores especiales de herencia

En el caso de que `.child` no tuviera la propiedad `border` establecida (o la tuviera a `initial` o `unset`), dicho `.child` no tendría borde ya que se trata de una propiedad no heredable. Sin embargo, si utilizamos el valor `inherit`, la propiedad hereda el valor de la propiedad `border` de su padre, y pasará a tener un borde propio, al igual que su elemento padre.

```
.container {  
  color: green;  
  border: 1px solid red;  
}  
  
.child {  
  border: inherit;  
}
```

# Herencia

## Valores especiales de herencia

```
.container {  
  color: green;  
  border: 1px solid red;  
}  
  
.child {  
  color: inherit;  
}
```

Por defecto, `.child` muestra el mismo color de texto que el elemento padre, ya que hereda su valor de color. Ocurre lo mismo si utilizamos el valor `inherit` o el valor `unset`. Sin embargo, si utilizamos el valor `initial`, pasa a utilizar un color negro para el texto, ya que es su valor inicial por defecto.



# Especificidad

# Especificidad

Todavía no sabemos seleccionar

# Especificidad

Volveremos a este tema en unas  
clases



**!important**

!important

Cuidado



# !important

Un buen diseño debe contener los mínimos  
!important posibles y, si se puede,  
ninguno

# ◀ Despedida ▶

Email

**bienvenidosaez@gmail.com**

Instagram

**@bienvenidosaez**

Youtube

**youtube.com/bienvenidosaez**

**CÓNQUER BLOCKS**